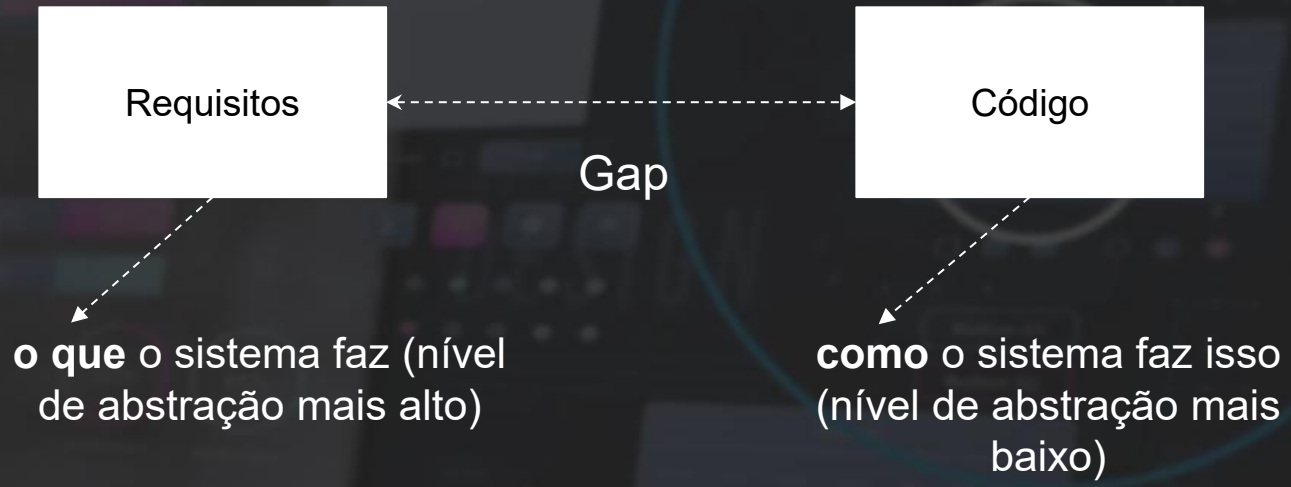
Modelos

FURB
UNIVERSIDADE DE BLUMENAU

Motivação

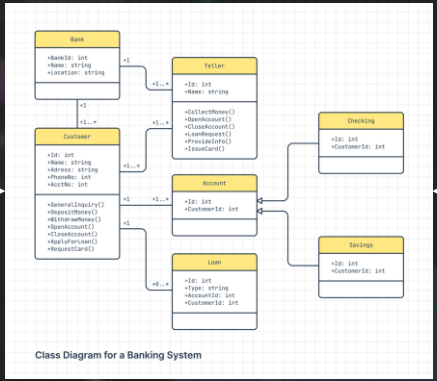
- Existe uma lacuna entre os seguintes mundos



Modelos de Software

- Objetivo: preencher essa "lacuna"
- Documentar uma solução para o problema definido pelos requisitos

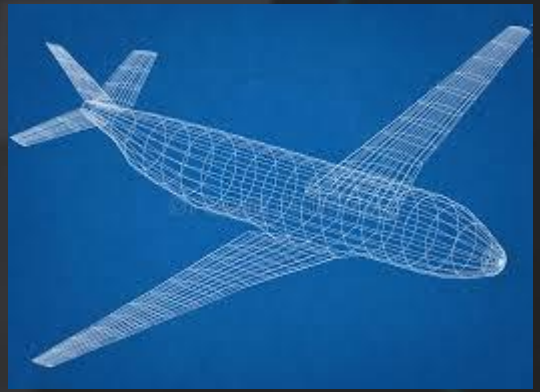
Requisitos

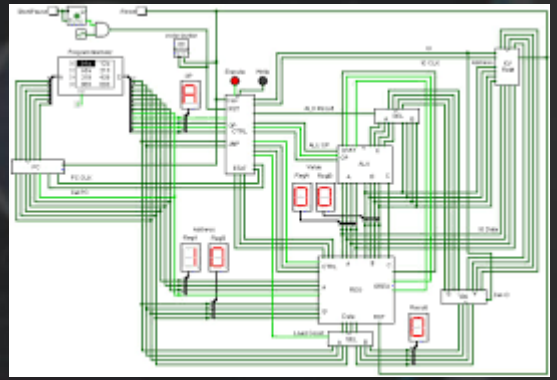


Código

Comuns em outras Engenharias

- Natural que fossem propostos também para software





Modelos de Software

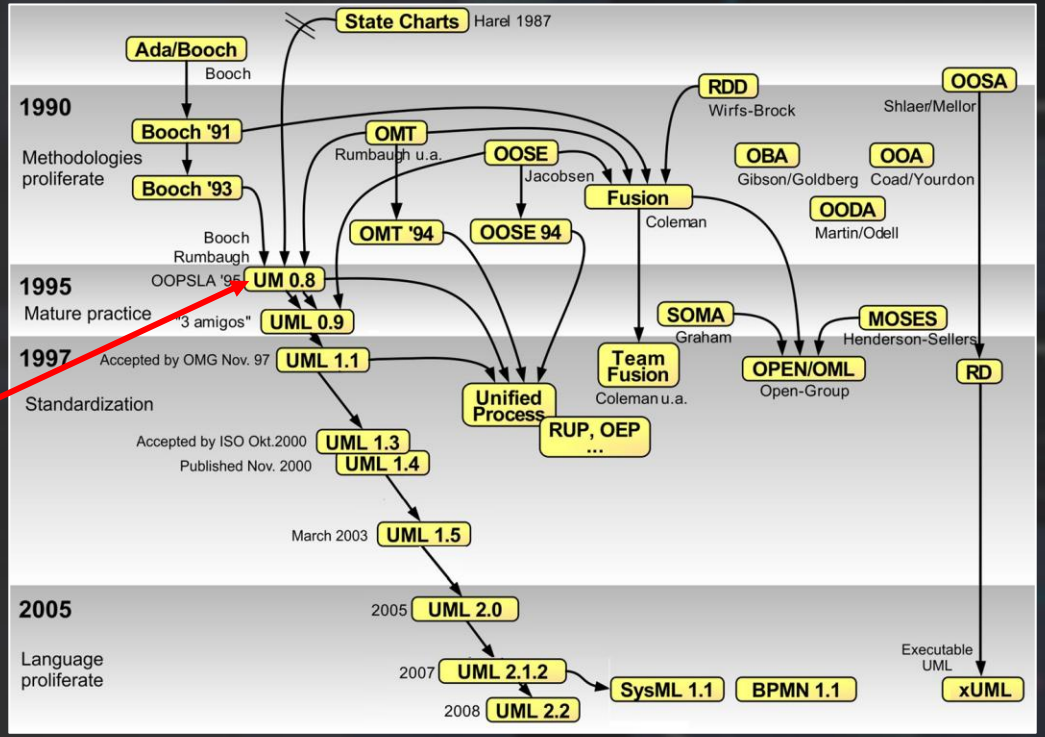
- Infelizmente, não são tão efetivos e largamente usados, como em outras engenharias
- Modelos de software podem ser:
  - Formais: menos comuns; não serão estudados aqui
  - Gráficos: UML é a notação mais comum

Ideia: "muitas vezes, um diagrama UML, vale mais do que 1000 linhas de código"

FURB
UNIVERSIDADE DE BLUMENAU

UML: Unified Modelling Language

- Proposta em 1995, para fundir outras notações



Fonte: Wikipédia

Quais os principais usos de UML?

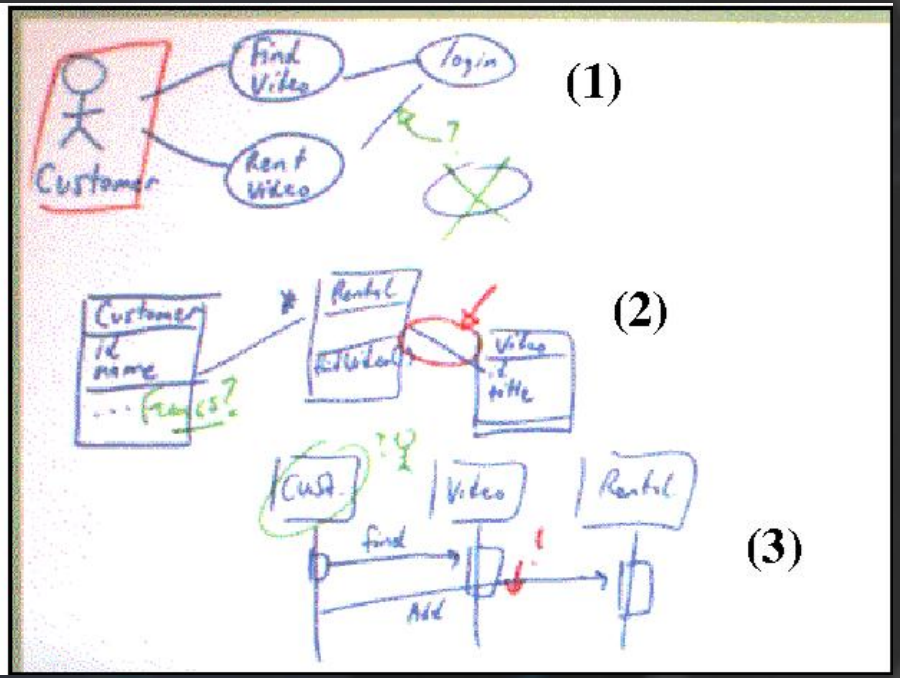
1. Como blueprint (planta detalhada)
2. Como sketches (esboços, rascunhos)

Vamos estudar - UML como Sketch

- Uso mais comum de UML com métodos ágeis
- Uso mais informal e leve da notação
- Objetivo **não** é ter um modelo completo
- UML é usada para:
  - Conversar sobre uma parte do código ou do projeto
  - Documentar uma parte do código ou do projeto

FURB
UNIVERSIDADE DE BLUMENAU

UML como Sketch



Q. Chen, J. Grundy, J. Hosking: SUMLOW: early design-stage sketching of UML diagrams on an E-whiteboard. Software Practice and Experience, 2008

Diagramas UML

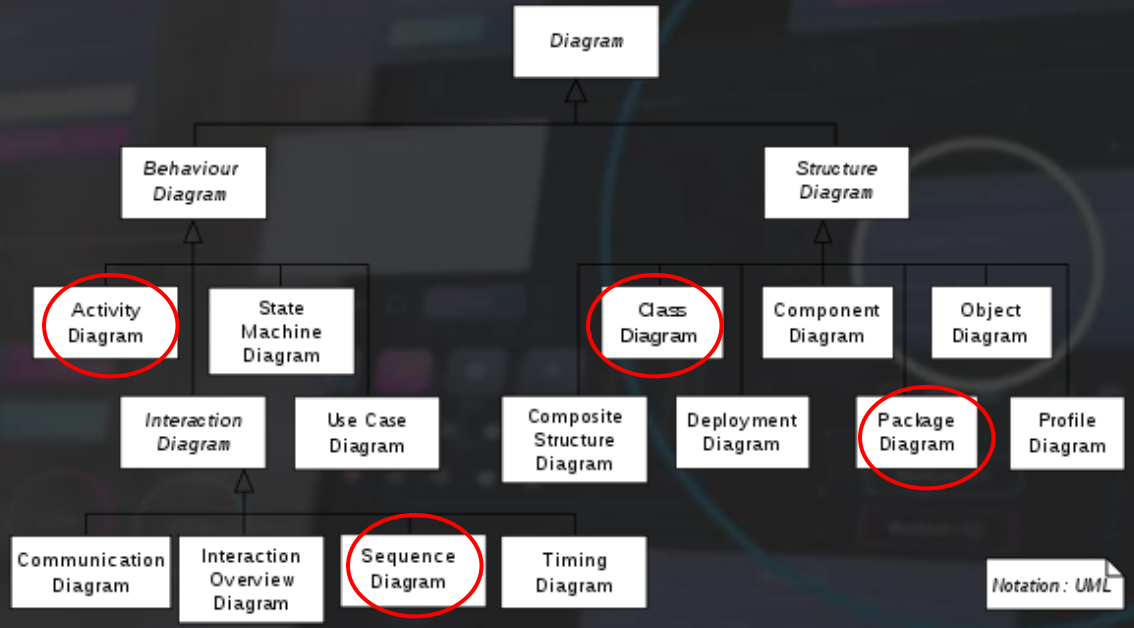
FURB
UNIVERSIDADE DE BLUMENAU

Diagramas UML

- Diagramas Estáticos: modelam a estrutura do código
- Diagramas Dinâmicos: modelam a execução do código (o comportamento do sistema)

Diagramas UML

Em vermelho, os diagramas que vamos estudar



Versão de UML que iremos usar

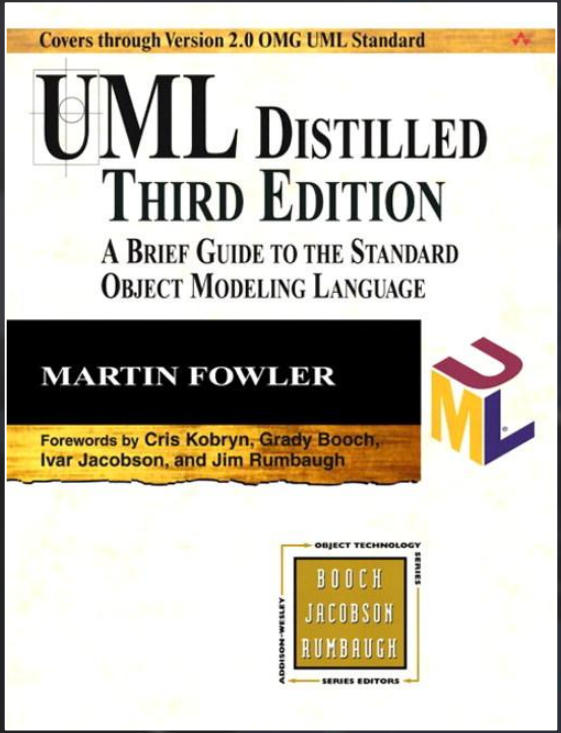


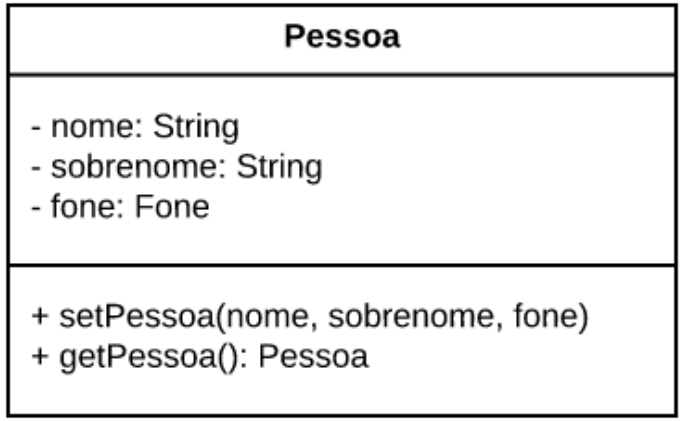
Diagrama de Classes

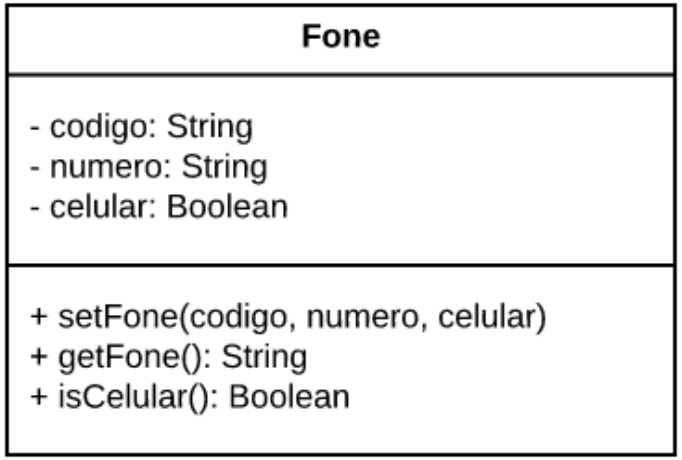
FURB
UNIVERSIDADE DE BLUMENAU

Formato genérico

[nome da classe]
[atributos]
[métodos]

Exemplo com duas classes

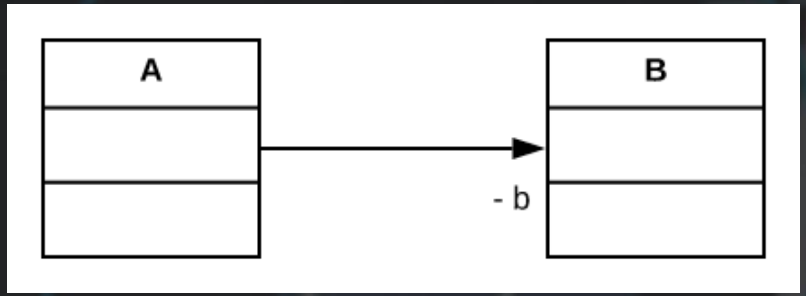




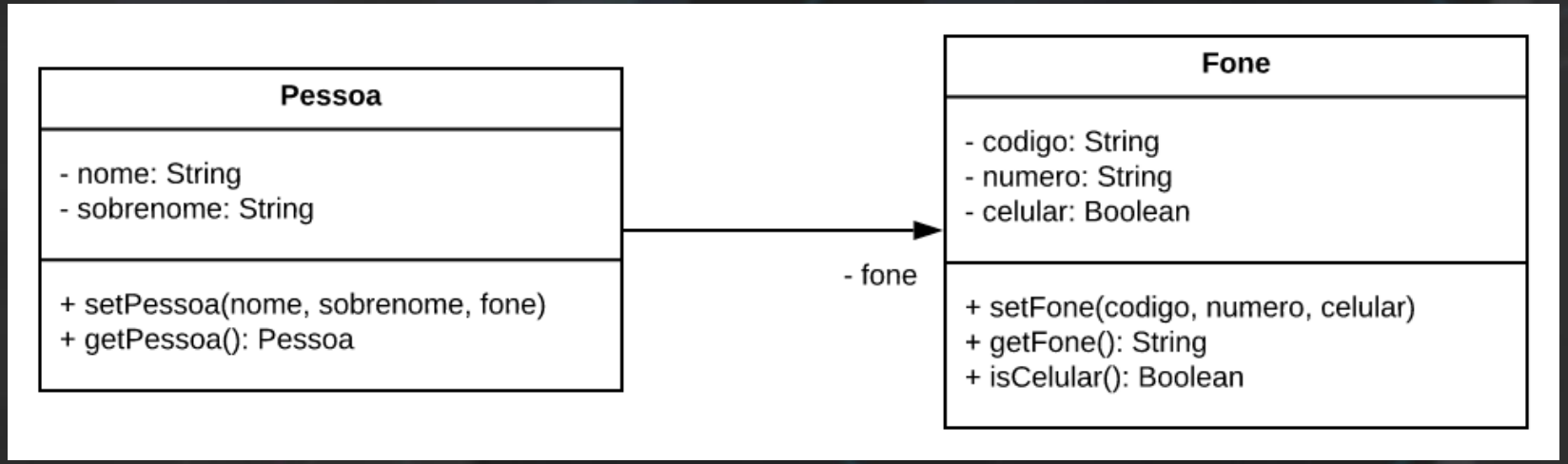
- : private
+: public

Associações

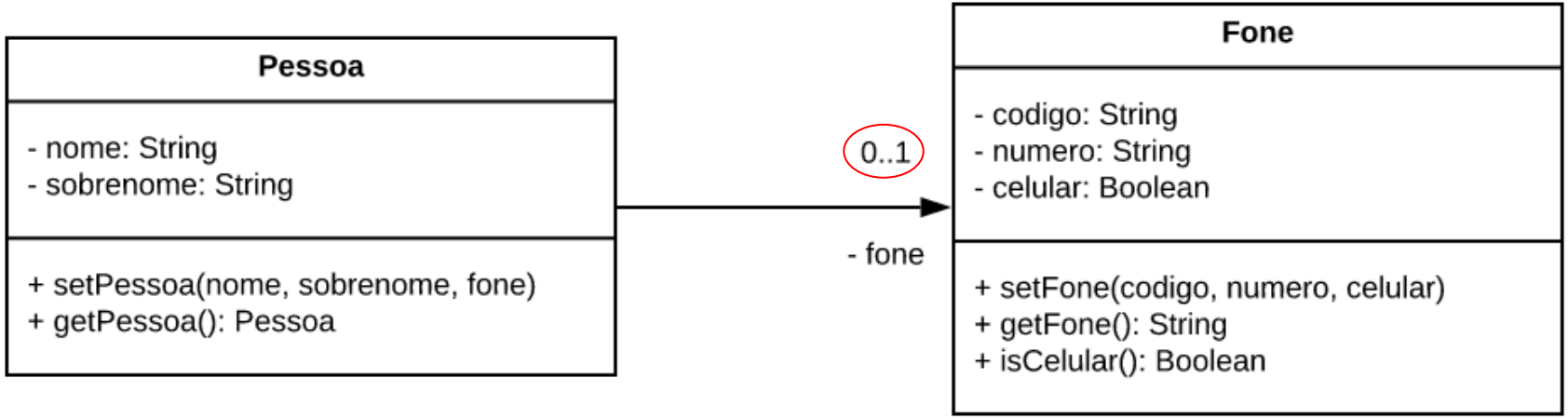
```
class A {
    ...
    private B b;
    ...
}


class B {
    ...
}
```



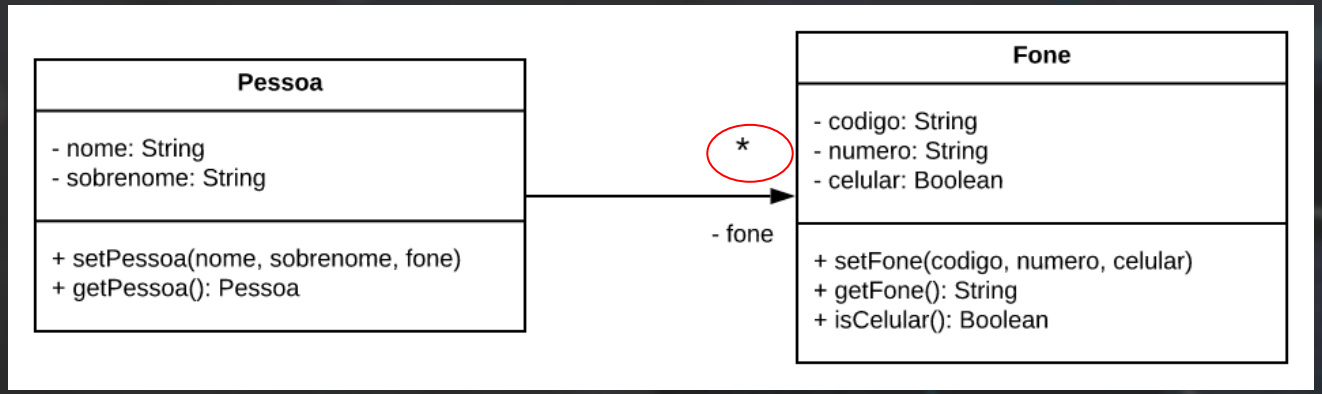
Associações



Multiplicidade (exemplo 1)



Multiplicidade (exemplo 2)




```
class Pessoa {
    private Fone[] fone;
    ...
}
class Fone {
    ...
}
```

Principais multiplicidades

0..1

1

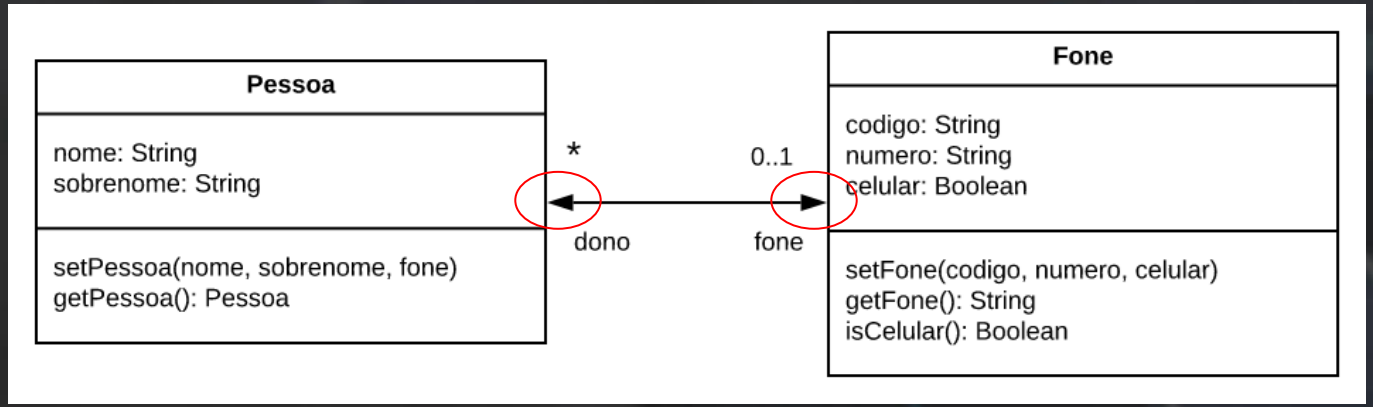
*

0..*

1..*

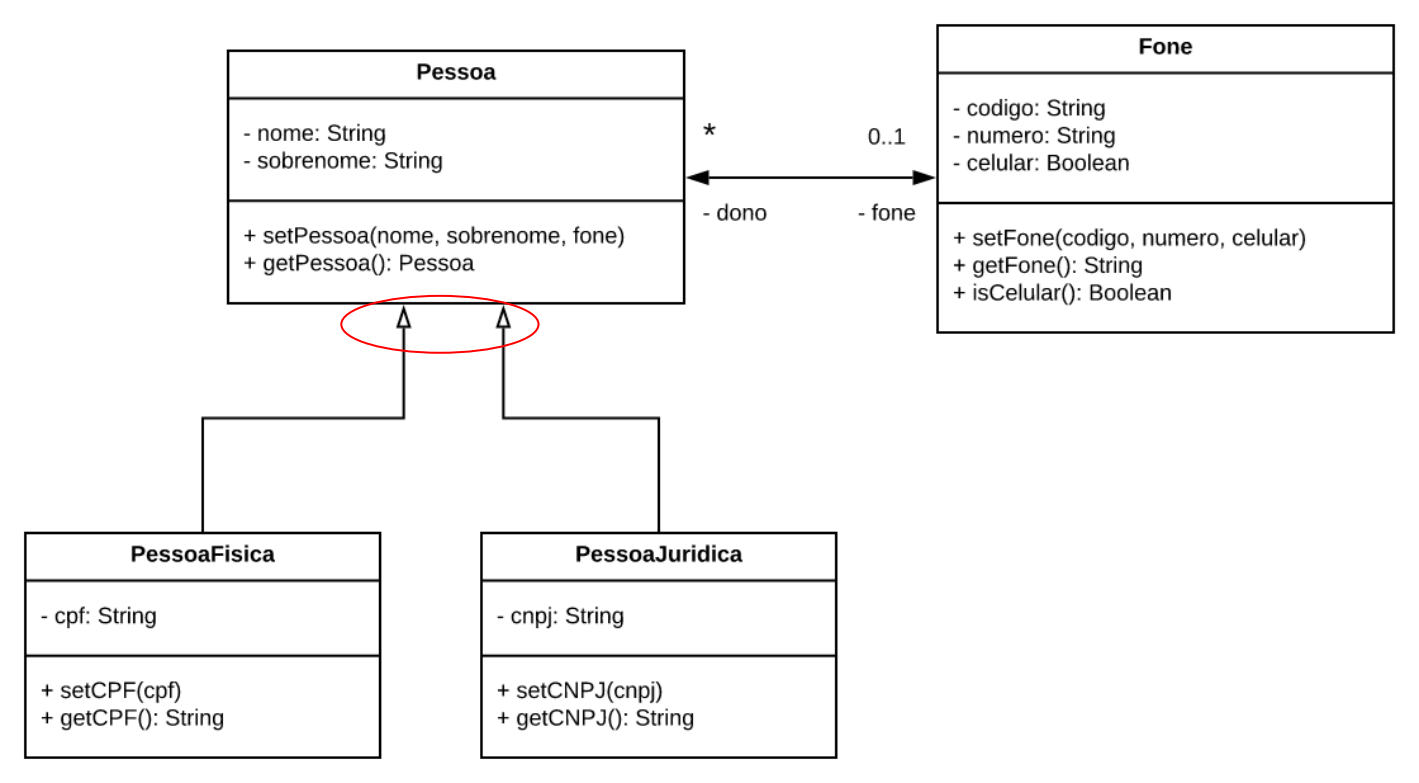
n

Associação bidirecional




```
class Pessoa {
    ...
    private Fone fone;
    ...
}

class Fone {
    ...
    private Pessoa[] dono;
    ...
}
```

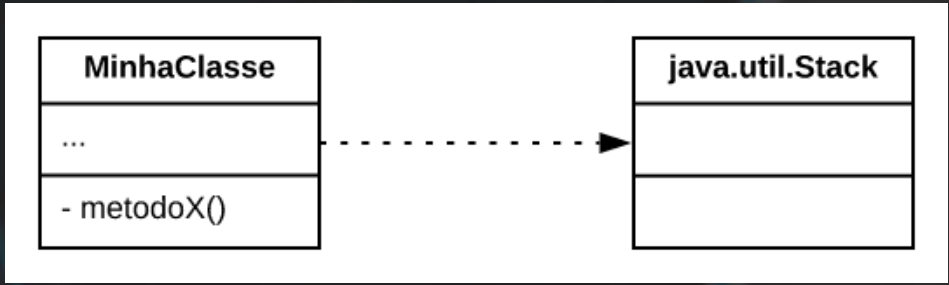
Herança


Pessoa
- nome: String
- sobrenome: String
+ setPessoa(nome, sobrenome, fone)
+ getPessoa(): Pessoa
*
0..1
- dono
- fone
Fone
- codigo: String
- numero: String
- celular: Boolean
+ setFone(codigo, numero, celular)
+ getFone(): String
+ isCelular(): Boolean
PessoaFisica
- cpf: String
+ setCPF(cpf)
+ getCPF(): String
PessoaJuridica
- cnpj: String
+ setCNPJ(cnpj)
+ getCNPJ(): String

Dependências (setas tracejadas)

Relacionamento entre duas classes, mas que não é devido a associação ou herança

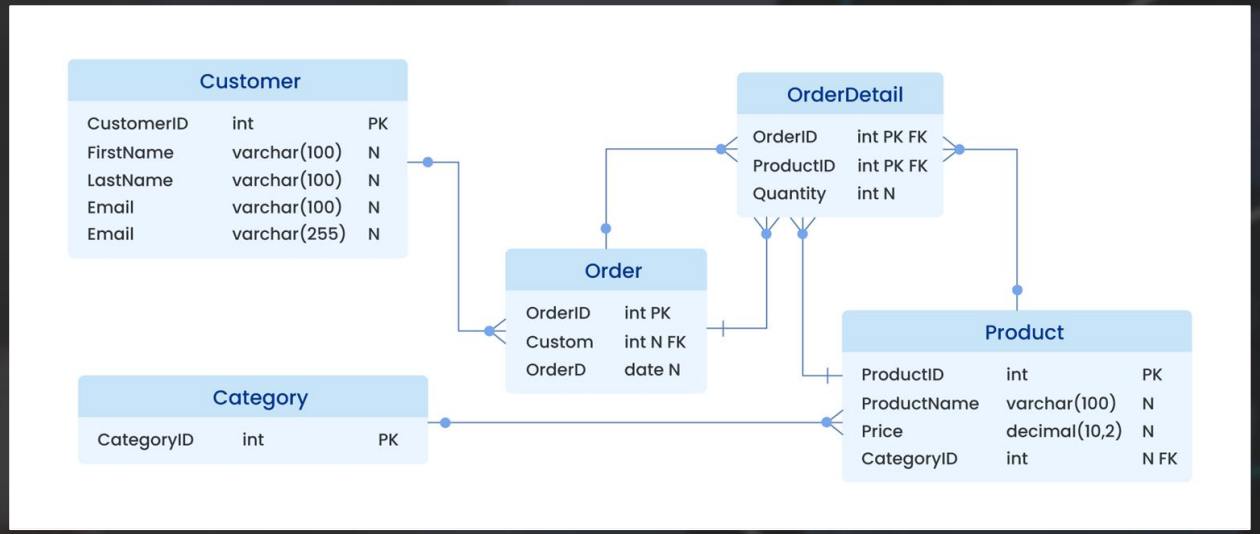
```
import java.util.Stack;

class MinhaClasse {
    ...
    private void metodoX() {
      Stack stack = new Stack();
      ...
    } ...
}
```



Dependências não possuem informação sobre multiplicidade

Importante

- Não confundir diagramas de classes com diagramas ER ou Esquemas de Bancos de Dados

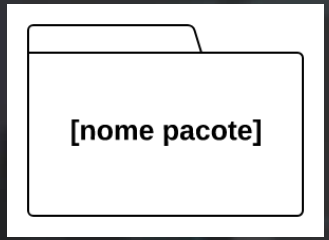


Não modela: métodos, herança, dependências, etc

Diagrama de Pacotes

FURB
UNIVERSIDADE DE BLUMENAU

Diagrama de Pacotes



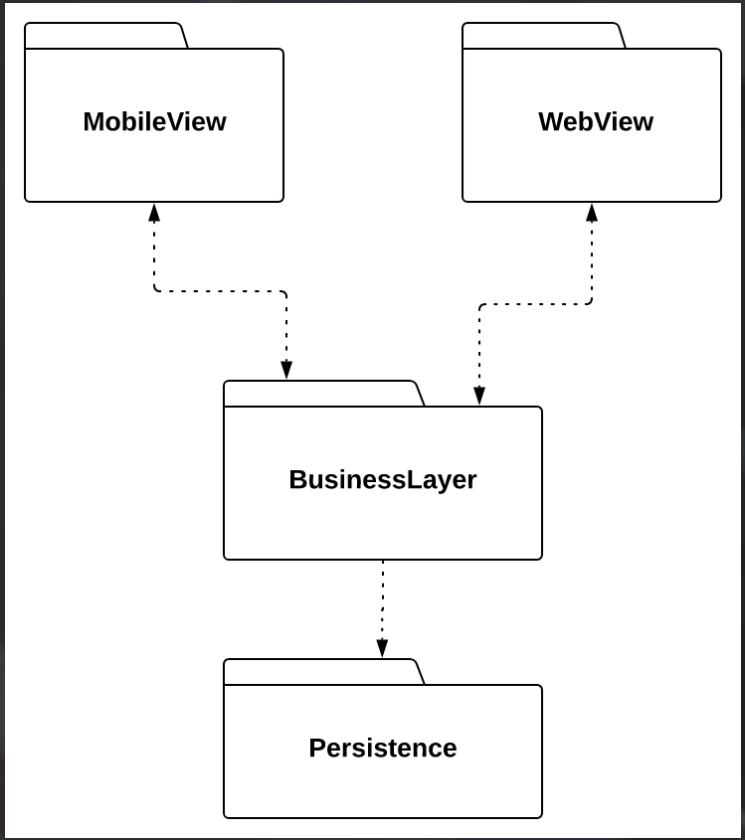
Usado para representar:

- Organização do sistema em **módulos**
- Dependências entre pacotes
- Camadas da aplicação
- Separação de responsabilidades

Quando é útil:

- Discutir arquitetura em reuniões
- Mostrar divisão em camadas (Controller, Service, Repository)
- Organizar módulos de um sistema grande
- Analisar dependências indevidas
- Detectar acoplamento entre camadas

Diagrama de Pacotes



Sketches UML são úteis em Engenharia Avante e em Engenharia Reversa

FURB
UNIVERSIDADE DE BLUMENAU

Engenharia Avante (“Forward”)

- Modelo é usado para discutir alternativas de projeto
- Antes de qualquer linha de código ser implementada

Engenharia Reversa

- Modelo é usado para explicar um código que já existe
- Contextos de manutenção e evolução de software

Diagrama de Sequência

FURB
UNIVERSIDADE DE BLUMENAU

Diagramas de Sequência

- São diagramas comportamentais ou dinâmicos
- Modelam:
  - Objetos de um sistema
  - Métodos que eles executam

Diagrama de Sequência (exemplo 1)

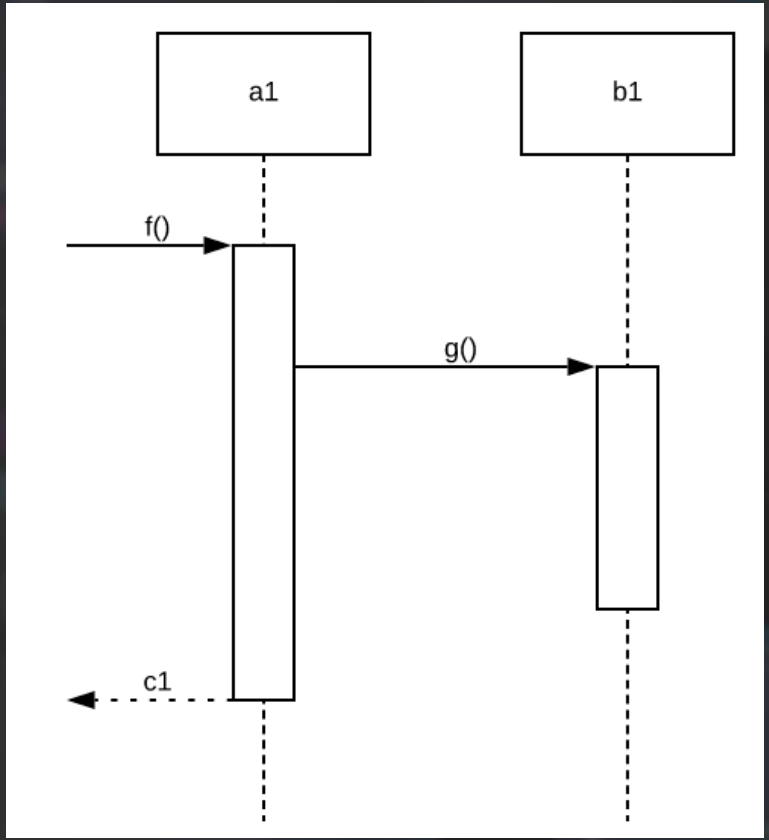
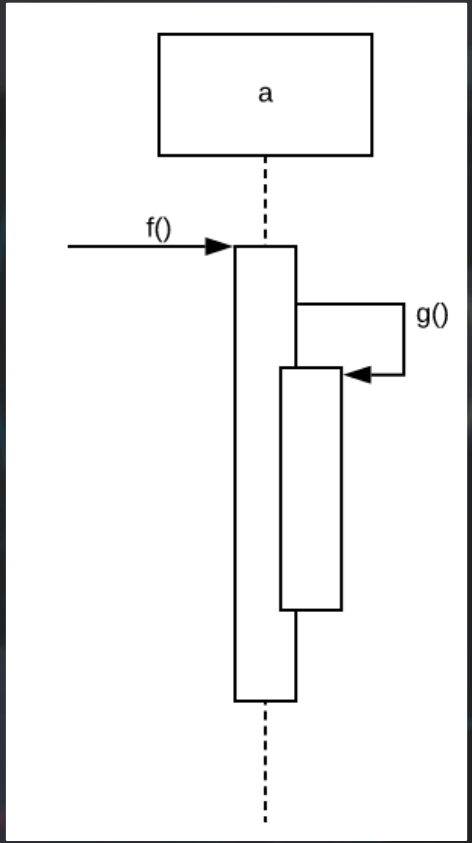


Diagrama de Sequência (exemplo 2)

```
class A {

  void g() {
    ...
  }

  void f() {
    ...
    g();
    ...
  }

  main() {
    A a = new A();
    a.f();
  }
}
```



Setas de retorno de métodos

- Muitas vezes, são omitidas:
  - Porque o retorno não é importante
  - Porque o método não retorna valor (void)

Diagrama de Sequência (exemplo 3)

O diagrama mostra um cenário mais real, que ilustra os métodos chamados quando o cliente de uma caixa eletrônico solicita um depósito de certo valor em sua conta.

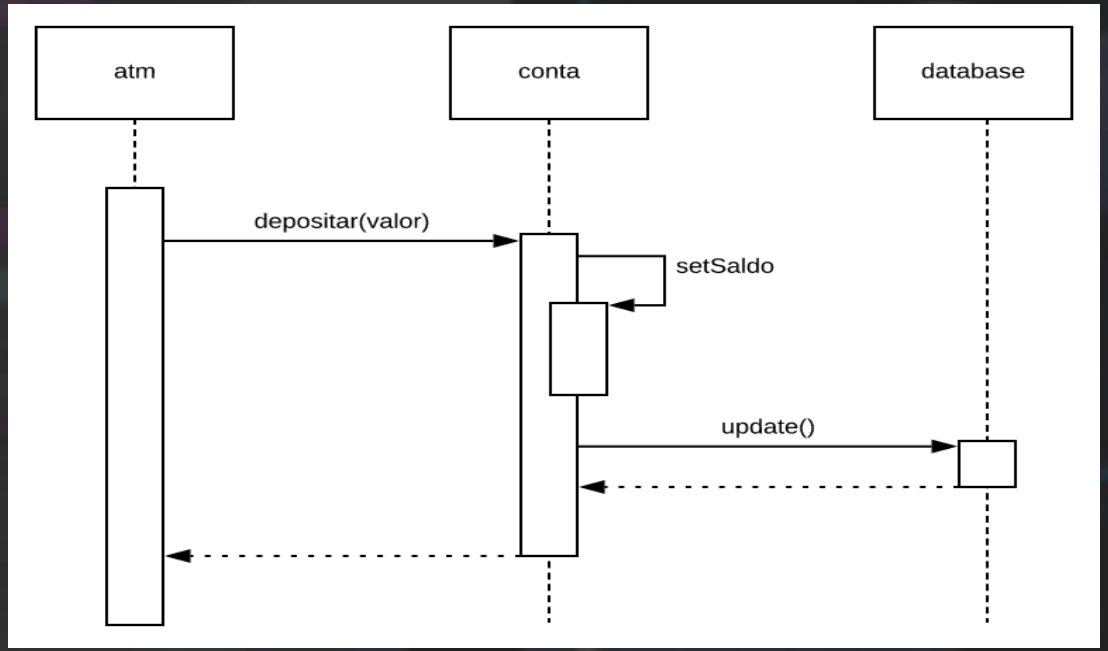
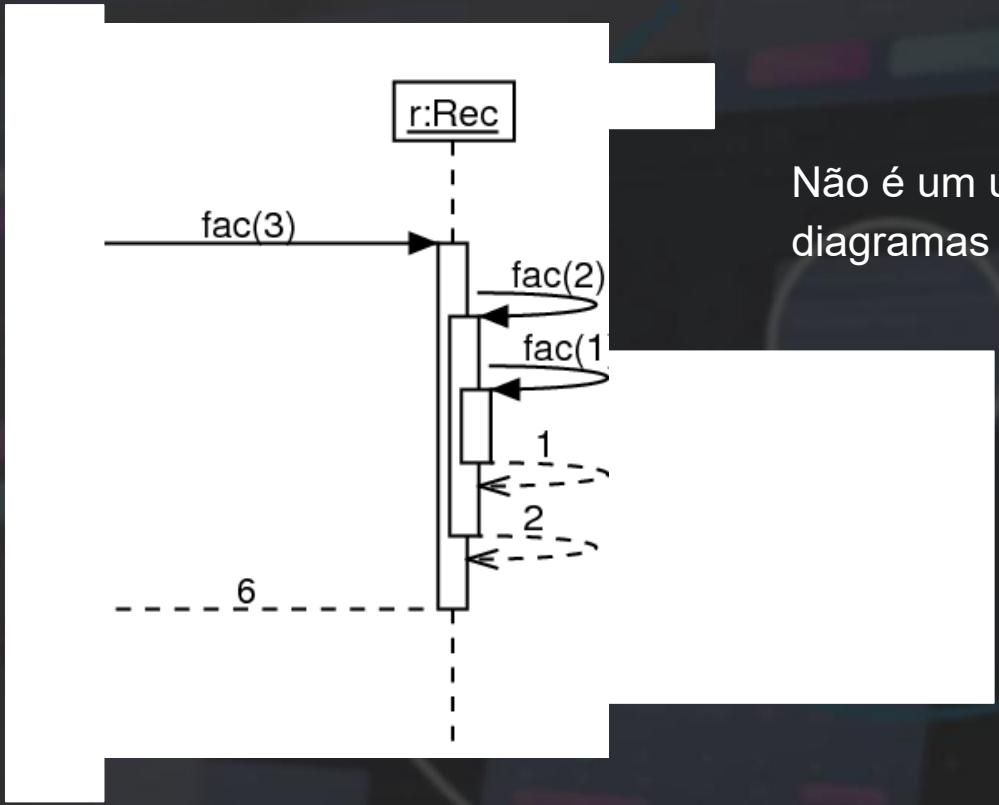


Diagrama de Sequência (exemplo 4)



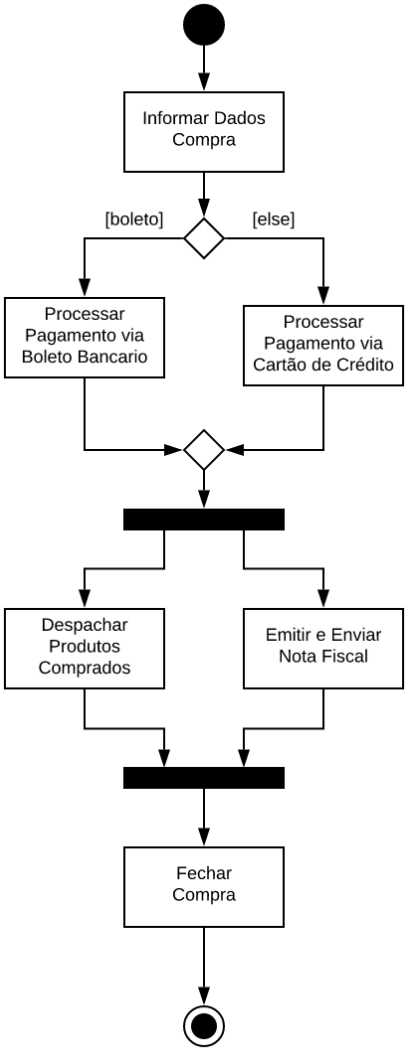
Não é um uso interessante de diagramas de sequência

Diagrama de Atividades

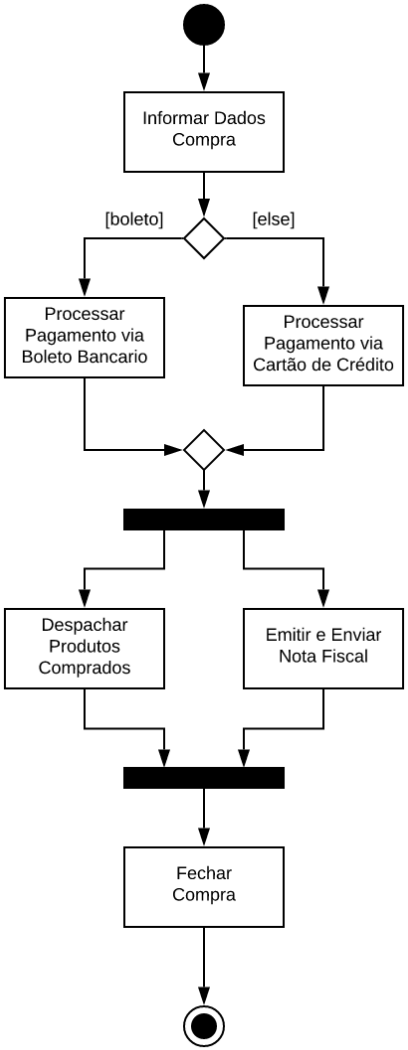
FURB
UNIVERSIDADE DE BLUMENAU

Diagramas de Atividades

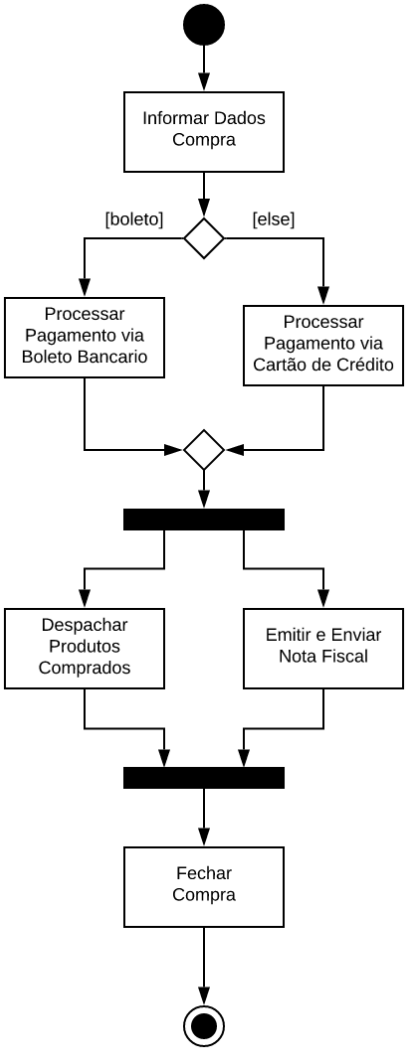
- Também são diagramas comportamentais ou dinâmicos
- Modelam em alto nível um processo ou fluxo de negócio



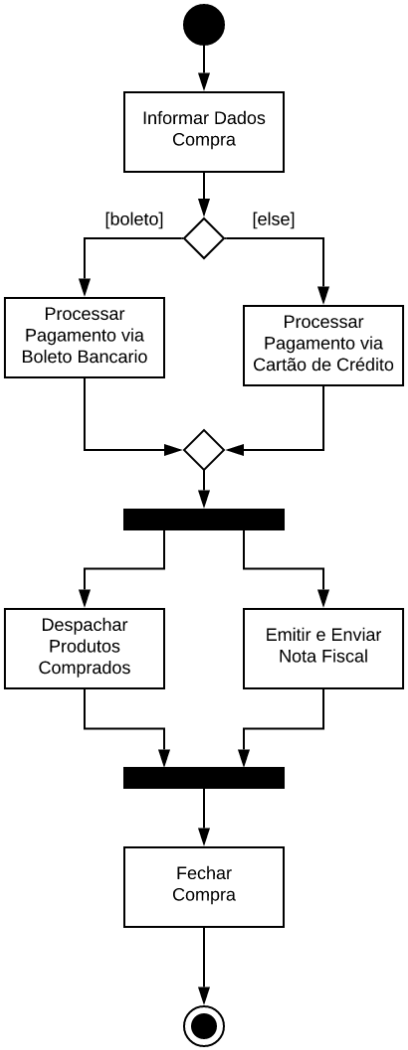
Imagine que existe uma ficha (token) que caminha pelos nodos do diagrama de atividades.



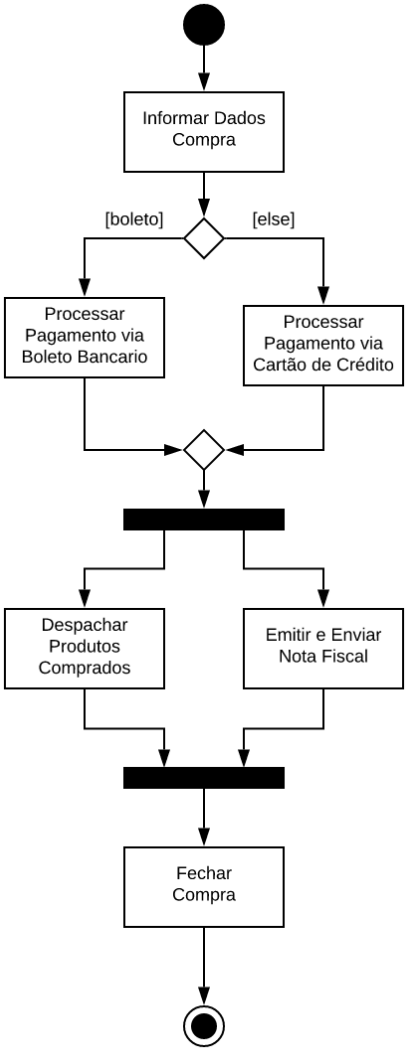
Nodo inicial (cria uma ficha)



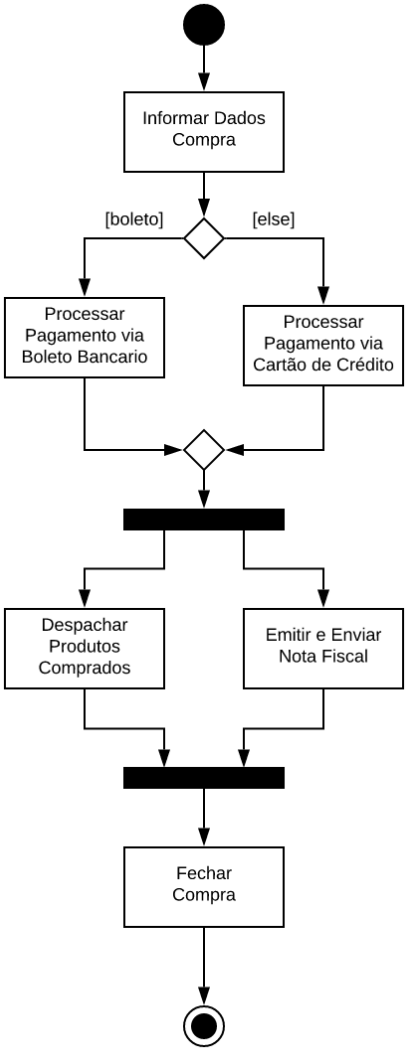
Ação (repassa ficha do fluxo de entrada para saída)



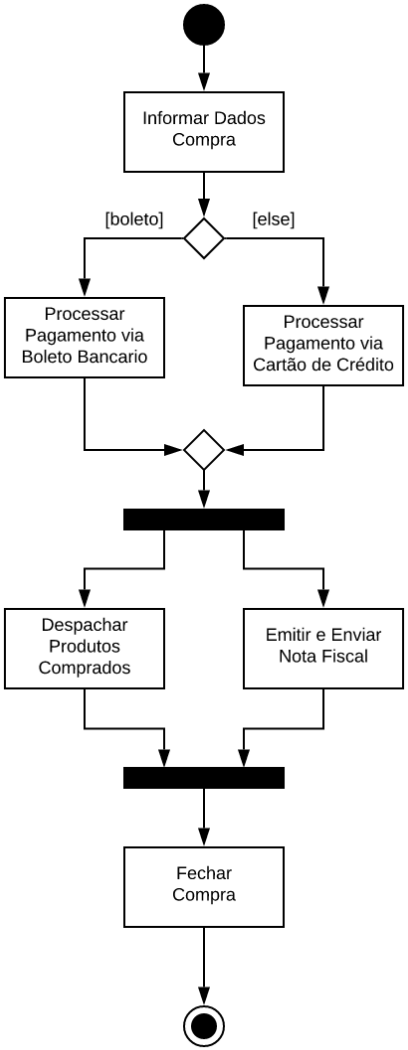
Decisão (decide para qual fluxo de saída repassará a ficha)



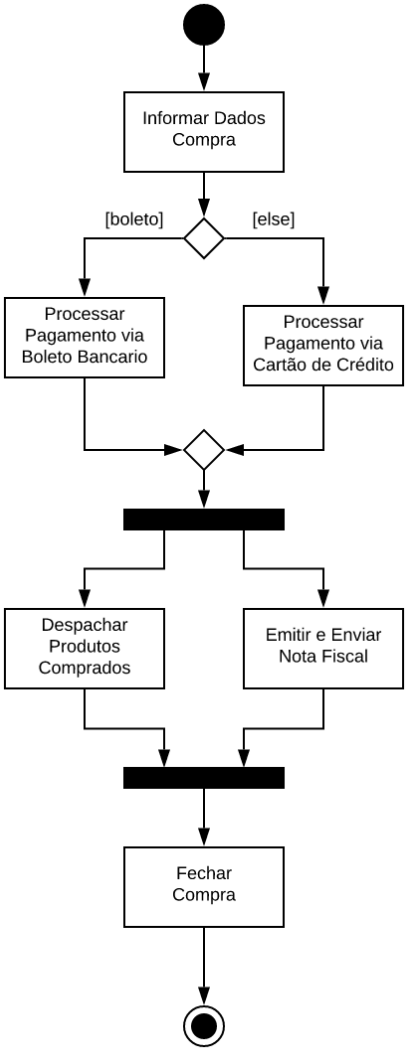
Merge (quando ficha chega em uma das entradas, repassa para saída)



Fork (multiplicadores de fichas)



Join (sorvedouros de fichas)



Nodo final (encerra execução)

Importante

- Diagramas de Atividades: modelam processos de mais alto nível de uma organização
- Não devem ser usados para modelar:
  - A navegação entre as páginas de um sistema
  - A implementação de uma parte de um sistema (isto é, não são um fluxograma)

Exercícios